Algorithms

Mihir Rao

Based on Algorithm Design by Kleinberg & Tardos

Contents

1 Introduction: Some Representative Problems **2**
1.1 Stable matching . . . 2
1.2 The Gale–Shapley algorithm . . . 3
1.3 Why the algorithm works . . . 4
1.4 Which stable matching does it find? . . . 5
1.5 Five representative problems . . . 6

2 Basics of Algorithm Analysis **10**
2.1 Computational tractability . . . 10
2.2 Asymptotic order of growth . . . 11
2.3 Implementing stable matching with lists and arrays . . . 13
2.4 A survey of common running times . . . 15
2.5 Priority queues and heaps . . . 18

3 Graphs **22**
3.1 Basic definitions and applications . . . 22
3.2 Connectivity and graph traversal . . . 23
3.3 Implementing graph traversal . . . 26
3.4 Testing bipartiteness . . . 29
3.5 Connectivity in directed graphs . . . 30
3.6 Directed acyclic graphs and topological ordering . . . 31

Chapter 1: Introduction: Some Representative Problems

The chapter introduces three parts of algorithm design: formulate the problem, construct an algorithm, and prove that it is correct and efficient. Stable matching develops this process in detail. Five further problems show how a small change in a problem's rules can require a different algorithmic approach.

1.1 Stable matching

An assignment of applicants to employers should be *self-enforcing*. If an applicant and an employer both prefer one another to their assigned partners, they have a reason to abandon the assignment. Stable matching rules out every such pair.

1.1.1 The model

Use the book's simplified setting:

- There are n men, $M = \{m_1, \ldots, m_n\}$, and n women, $W = \{w_1, \ldots, w_n\}$.
- Each person ranks every person on the other side, with no ties.
- Each person can have at most one partner.

Definition 1.1 [Matching and stability]. A matching is a set $S \subseteq M \times W$ in which no person appears twice. It is perfect when everyone appears exactly once.

For a perfect matching containing (m, w) and (m', w'), an unmatched pair (m, w') is an instability when

$$m \text{ prefers } w' \text{ to } w, \qquad w' \text{ prefers } m \text{ to } m'.$$

A matching is stable if it is perfect and has no instability.

Both people must prefer the change. One person's dissatisfaction alone does not make a pair unstable.

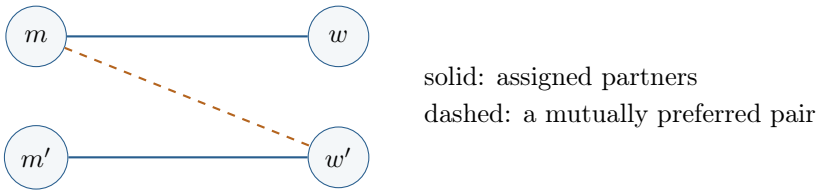


Figure 1: An instability in a perfect matching. Both m and w' would leave their partners to form the dashed pair.

1.1.2 The book's two small examples

Example 1: a unique stable matching. Let $M = \{m, m'\}$ and $W = \{w, w'\}$. Everyone agrees on the ordering of the other side:

man	preference	woman	preference
m	$w \succ w'$	w	$m \succ m'$
m'	$w \succ w'$	w'	$m \succ m'$

Here $a \succ b$ means that a is preferred to b. There are only two perfect matchings to check.

1. In $S_1 = \{(m, w), (m', w')\}$, m and w have their first choices. Neither would leave for the other person, so no unmatched pair can block S_1.
2. In $S_2 = \{(m, w'), (m', w)\}$, m and w both prefer each other to their assigned partners. The pair (m, w) blocks S_2.

Thus S_1 is the unique stable matching.

Example 2: two stable matchings. Now use these preferences:

man	preference	woman	preference
m	$w \succ w'$	w	$m' \succ m$
m'	$w' \succ w$	w'	$m \succ m'$

The same two perfect matchings are now both stable:

- $S_1 = \{(m, w), (m', w')\}$ gives both men their first choices. Every unmatched pair contains a man who prefers his current partner.
- $S_2 = \{(m, w'), (m', w)\}$ gives both women their first choices. Every unmatched pair contains a woman who prefers her current partner.

Stability therefore need not determine a unique answer. This example will also explain why the proposing side matters.

1.2 The Gale–Shapley algorithm

A proposal creates a tentative engagement. A woman keeps the best proposal she has received so far, but may later replace her partner. A free man proposes to his highest-ranked woman who has not already rejected him.

Algorithm 1.1: Gale–Shapley, with men proposing

```
Input: Complete, strict preference lists for M and W, with |M| = |W| = n
Mark everyone free
while some man m is free and has not proposed to every woman do
    Choose such a man m
    Let w be his highest-ranked woman not yet proposed to
    Record the proposal from m to w
    if w is free then
        Engage m and w
    else
        Let m′ be w's current partner
        if w prefers m to m′ then
            Engage m and w; make m′ free
        else
            Leave m free
return the set S of engaged pairs
```

For Example 1, one possible execution starts with m' rather than m:

Step	Proposal	Result
1	$m' \to w$	w accepts m'
2	$m \to w$	w keeps m; m' becomes free
3	$m' \to w'$	w' accepts m'

The first engagement was temporary. The final matching is S_1, as required.

1.3 Why the algorithm works

Two observations drive the proof:

- Once a woman receives a proposal, she remains engaged. Her partner can only improve in her preference order.
- A man's proposals move down his preference list. He never proposes to the same woman twice.

The first follows from the rule for accepting a new proposal; the second follows from always choosing the best untried woman.

Theorem 1.2 [Termination and correctness]. *Gale–Shapley terminates after at most n^2 proposals and returns a stable matching.*

Proof.

1. **Termination.** Each iteration adds a new proposed pair to a set $P(t) \subseteq M \times W$. Hence $|P(t)|$ increases by one and never exceeds n^2. The number of free people would not be a suitable progress measure, since replacing a partner may leave that number unchanged.

2. **Perfection.** A free man cannot have proposed to all n women. If he had, all n women would have received a proposal and would still be engaged. Their distinct partners would require n engaged men in addition to the free man, although only n men exist.

   Therefore, if a man were free at termination, he would have an untried woman and the loop could continue. Everyone must be matched when it stops.

3. **Stability.** Suppose the final pairs are (m, w) and (m', w'), but m and w' prefer each other. Since m prefers w' to w, he must have proposed to w' earlier and been rejected, either immediately or after an engagement. At that rejection, w' kept someone she preferred to m. Her partner could only improve afterward, so she prefers her final partner m' to m. This contradicts the proposed instability.

□

The bound counts proposals. An implementation also needs efficient ways to find a free man, advance his list, and compare two ranks; those details are developed in Chapter 2.

1.4 Which stable matching does it find?

The algorithm may choose any free man at each step. Nevertheless, all such executions give the same result when men propose.

Definition 1.3 [Valid partners]. A woman w is a valid partner of m if some stable matching contains (m, w). Let $\text{best}(m)$ be m's highest-ranked valid partner, and define

$$S^* = \{(m, \text{best}(m)) : m \in M\}.$$

A best valid partner is the best attainable *among stable matchings*. She need not be the first woman on the original preference list.

Theorem 1.4 [Proposer optimality]. *Every execution of men-proposing Gale–Shapley returns S^*.*

Proof. Assume that some execution rejects a man from a valid partner. Consider the first such rejection: w rejects m in favor of m', whom she prefers.

1. Because w is valid for m, there is a stable matching S' containing (m, w). Write (m', w') for the pair containing m' in S'; here $w' \neq w$.
2. The woman w' is a valid partner of m'. If m' preferred w' to w, he would have proposed to w' first. He could reach w only after being rejected by w', an earlier rejection by a valid partner. This would contradict the choice of the first rejection. Thus m' prefers w to w'.
3. In S', m' prefers w to his partner w', and w prefers m' to her partner m. The pair (m', w) blocks S', contradicting stability.

No man can therefore be rejected by a valid partner. Since he proposes in preference order and the final matching is stable, each man finishes with his best valid partner. □

This proves both that S^* is a matching and that proposal order does not change the result. In Example 2, men proposing produces S_1; women proposing produces S_2.

Proposition 1.5 [The receiving side]. *In S^*, every woman receives her worst valid partner.*

Proof. Suppose S^* contains (m, w), but another stable matching S' pairs w with m', whom she likes less than m. Let w' be m's partner in S'. Since $w = \text{best}(m)$ and w' is also valid, m prefers w to w'. Thus (m, w) blocks S', a contradiction. □

The guarantee concerns valid partners, not all partners on the preference list. Reversing the proposing side reverses these two guarantees.

1.5 Five representative problems

A graph $G = (V, E)$ records pairwise relationships: V is a set of nodes and an edge $\{u, v\} \in E$ joins two nodes. The five problems use related selection and assignment ideas, but have different structures.

1.5.1 Interval scheduling

Request i uses one resource from start time s_i to finish time f_i, where $s_i < f_i$. Two requests are compatible when

$$f_i \leq s_j \quad \text{or} \quad f_j \leq s_i.$$

A finish may coincide with the next start.

- **Goal.** Choose a compatible set A maximizing $|A|$.
- **Approach.** A suitable greedy rule processes requests in order and makes irrevocable choices. The rule and its proof come later.

1.5.2 Weighted interval scheduling

Each request now has a value $v_i > 0$. The compatibility condition stays the same, but the objective becomes

$$\max \left\{ \sum_{i \in A} v_i : A \text{ is compatible} \right\}.$$

For the book's example, suppose $v_1 > \sum_{i=2}^{n} v_i$. Every solution that excludes request 1 is worth less than request 1 alone. An optimum must therefore include request 1, regardless of its conflicts with other requests.

Values change which choices are sensible. Dynamic programming will compare alternatives through a table of optimal subproblem values. Setting every $v_i = 1$ recovers ordinary interval scheduling.

1.5.3 Bipartite matching

A graph is BIPARTITE if its nodes split into X and Y, and every edge joins the two sides. A matching is a set of edges with no shared ends; a perfect matching covers every node once.

- **Model.** Nodes in X may be jobs and nodes in Y machines. An edge means that the machine can perform the job.
- **Goal.** Find a matching of maximum size. If $|X| = |Y| = n$, a perfect matching exists exactly when that maximum is n.
- **Approach.** Augmentation enlarges a matching by revising some previous choices. It leads to the network-flow methods developed later.

This problem has allowed edges but no preference lists. Its objective is to match as many pairs as possible, rather than to prevent an instability.

1.5.4 Independent set

A node set $S \subseteq V$ is INDEPENDENT if no edge has both ends in S. The objective is to maximize $|S|$. An edge represents a conflict between two objects that cannot both be selected.

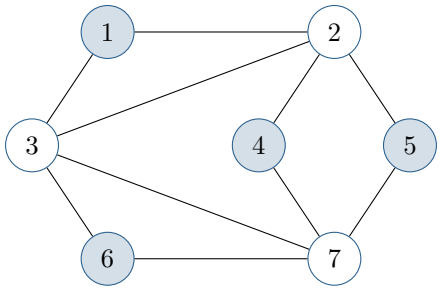


Figure 2: The book's independent-set example. The shaded nodes $\{1, 4, 5, 6\}$ have no edges between them.

The shaded set has size four. It is maximum: the triangle $\{1, 2, 3\}$ contributes at most one node, the edge $\{4, 7\}$ at most one, and the two remaining nodes $5, 6$ at most two.

Two earlier problems fit this model:

1. **Interval scheduling.** Make one node per interval and join intervals that overlap. Independent sets are exactly compatible schedules.
2. **Bipartite matching.** Given $G' = (V', E')$, make one node for *each edge* of G'. Join two new nodes when their original edges share an end. Independent sets in the new graph are exactly matchings in G'.

In the second construction, choosing nonadjacent new nodes means choosing original edges with no shared ends. This is why edges become nodes.

For general graphs, brute force examines all $2^{|V|}$ node subsets. The decision question, whether an independent set of size at least k exists, is NP-complete. A proposed set of size k is easy to verify: count its nodes and check that no pair is joined by an edge. This establishes a large enough independent set, not that the proposed set is maximum.

1.5.5 Competitive facility location

Two companies alternately open facilities in zones represented by a graph. Zone i has value b_i. Selecting a zone makes that node and its neighbors unavailable to both companies, so all selected zones together must form an independent set.

Player P_1 moves first. Given a target B, ask whether P_2 has a strategy that guarantees total value at least B, whatever P_1 does.

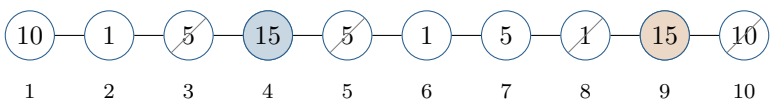


Figure 3: The book's zone values, with an illustrative legal sequence: P_1 selects zone 4 and P_2 selects zone 9. Node labels are values; small labels below are zone numbers. Crossed neighbors cannot be selected.

The displayed two moves are legal because zones 4 and 9 are not adjacent. They do not establish a winning strategy: P_1 could have chosen a different first zone. A strategy must specify responses to those different choices and to later moves.

This distinguishes the problem from verifying one independent set. A successful play is one sequence; a guaranteed outcome must handle opposing choices throughout the game. The chapter uses this distinction to introduce PSPACE-complete problems. These are believed to be harder than NP-complete problems, although that separation has not been proved.

Source. Chapter 1, Introduction: Some Representative Problems, *Algorithm Design*.

Chapter 2: Basics of Algorithm Analysis

Algorithm analysis asks how time and memory requirements grow with the input. The aim is to find structure that lets an algorithm avoid a large search space, then choose representations that make its individual operations efficient.

2.1 Computational tractability

2.1.1 Input size and worst-case time

Timing an implementation is useful, but its results depend on the machine, the code, and the particular test instances. A mathematical analysis describes how an algorithm scales across all inputs of a given size.

Let N be the size of the input representation. The WORST-CASE RUNNING TIME $T(N)$ is the largest number of steps taken on any input of size N. Average-case analysis instead needs a probability distribution on inputs; conclusions can change substantially with that choice.

The size parameter must be explicit. For stable matching with n people on each side, the $2n$ preference lists contain $N = 2n^2$ entries, ignoring the finer details of encoding each entry. An $O(n^2)$ algorithm is therefore linear in the number of preference entries. Yet there are $n!$ perfect matchings, so the input compactly describes an enormous search space.

We count basic operations such as array access, pointer traversal, assignment, comparison, and arithmetic on fixed-size integers as constant time. A high-level instruction that scans a whole list must be charged for that scan, rather than counted as one step.

2.1.2 Polynomial time as the benchmark

Improving substantially on brute-force search is a useful goal, but it does not give a precise definition of efficiency. The book uses polynomial time as its main formal benchmark.

Definition 2.1 [POLYNOMIAL TIME]. An algorithm has POLYNOMIAL RUNNING TIME if constants $c, d > 0$, independent of N, bound its running time on every input by cN^d. Equivalently, $T(N) = O(N^d)$ for some fixed d.

Doubling the input multiplies this bound by 2^d:

$$c(2N)^d = 2^d cN^d.$$

For an exponential bound 2^N, increasing N by one doubles the bound, and doubling N multiplies it by 2^N. These are very different scaling behaviors.

Example: To compare growth rates, suppose each listed operation takes one microsecond. For $n = 50$, n^3 operations take 0.125 seconds, whereas 2^n operations take about 35.7 years, using 365 days per year. These are illustrative operation counts, not measurements of an implementation.

Polynomial time is an abstraction of practical efficiency. A large exponent or constant can make a polynomial algorithm unusable; an exponential algorithm may work well on a restricted collection of inputs. Still, the benchmark often separates useful algorithms from infeasible exhaustive searches. It also makes the question of whether a problem admits an efficient algorithm mathematically precise.

2.2 Asymptotic order of growth

Exact step counts change with the level of implementation. Asymptotic bounds ignore fixed multiplicative factors and focus on growth for large inputs. Throughout this section, functions are nonnegative for sufficiently large n.

2.2.1 Upper, lower, and tight bounds

Definition 2.2 [ASYMPTOTIC BOUNDS]. Let T and f be functions of the input size.

- $T(n) = O(f(n))$ means that constants $c > 0$ and n_0 exist such that $T(n) \leq cf(n)$ for every $n \geq n_0$. This is an ASYMPTOTIC UPPER BOUND.
- $T(n) = \Omega(f(n))$ means that constants $c > 0$ and n_0 exist such that $T(n) \geq cf(n)$ for every $n \geq n_0$. This is an ASYMPTOTIC LOWER BOUND.
- $T(n) = \Theta(f(n))$ means both bounds hold: there are constants $c_1, c_2 > 0$ and n_0 such that

  $$c_1 f(n) \leq T(n) \leq c_2 f(n) \qquad (n \geq n_0).$$

  This is an ASYMPTOTICALLY TIGHT BOUND.

The constants are fixed; none may depend on n.

For example, let $T(n) = pn^2 + qn + r$ with $p, q, r > 0$. For $n \geq 1$,

$$pn^2 \leq T(n) \leq (p + q + r)n^2,$$

so $T(n) = \Theta(n^2)$. It is also $O(n^3)$ and $\Omega(n)$, but these weaker bounds do not identify its growth as precisely.

Note. O, Ω, and Θ describe bounds on a specified function. If T is a worst-case running time, all three describe that worst-case function; Ω does not mean that we have switched to best-case analysis.

Proposition 2.3 [A POSITIVE RATIO LIMIT]. *Suppose $g(n) > 0$ for sufficiently large n and*

$$\lim_{n\to\infty} \frac{f(n)}{g(n)} = c, \qquad 0 < c < \infty.$$

Then $f(n) = \Theta(g(n))$.

Proof. Beyond some n_0, the ratio lies between $c/2$ and $2c$. Multiplying by $g(n)$ gives $(c/2)g(n) \le f(n) \le 2cg(n)$, which supplies both bounds. □

2.2.2 Combining bounds

Two rules account for many running-time calculations.

1. **Transitivity.** If $f = O(g)$ and $g = O(h)$, then $f = O(h)$. Indeed, for n beyond both thresholds,

   $$f(n) \le cg(n) \le cc'h(n).$$

   The same argument with reversed inequalities proves transitivity for Ω. Applying both proves transitivity for Θ.

2. **Sums.** If $f = O(h)$ and $g = O(h)$, then $f + g = O(h)$, since

   $$f(n) + g(n) \le (c + c')h(n)$$

   beyond both thresholds. This extends to a fixed number of terms. If $g = O(f)$, nonnegativity also gives $f + g \ge f$, so

   $$f + g = \Theta(f).$$

   Thus a slower-growing phase does not change the order of the total time.

The number of terms must be fixed for the stated sum rule. Adding n constant-time operations gives $O(n)$ time, not $O(1)$.

2.2.3 Polynomials, logarithms, and exponentials

For a polynomial $f(n) = a_0 + a_1 n + \cdots + a_d n^d$ with $a_d > 0$,

$$\frac{f(n)}{n^d} = a_d + \frac{a_{d-1}}{n} + \cdots + \frac{a_0}{n^d} \longrightarrow a_d.$$

Hence $f(n) = \Theta(n^d)$, even if some lower coefficients are negative. Polynomial-time bounds also include fractional powers and bounds such as $n \log n$: for example, $n \log_2 n \le n^2$ for integer $n \ge 1$.

The logarithm $\log_b n$ is the exponent x such that $b^x = n$. For integer $n \ge 1$, its base-b digit count is $\lfloor \log_b n \rfloor + 1$ when b is an integer base. The change-of-base identity

$$\log_a n = \frac{\log_b n}{\log_b a}$$

shows that all fixed bases $a, b > 1$ give the same asymptotic order.

For every fixed $b, r > 1$ and $x, d > 0$,

$$\log_b n = O(n^x), \qquad n^d = O(r^n).$$

In fact the corresponding ratios tend to zero: every positive power eventually outgrows a logarithm, and every exponential with base greater than one eventually outgrows a fixed power.

The base of an exponential cannot be dropped in the same way. If $r > s > 1$, then $r^n / s^n = (r/s)^n \to \infty$, so r^n and s^n are not Θ of one another.

2.3 Implementing stable matching with lists and arrays

Chapter 1 bounded Gale–Shapley by n^2 proposals. To obtain an $O(n^2)$ running time, preprocessing must take $O(n^2)$ time and each proposal must take $O(1)$ time.

2.3.1 Choosing the representation

An array gives $O(1)$ access to its ith entry. Searching an unsorted array takes $O(n)$ time; a sorted array supports binary search in $O(\log n)$ time. Inserting or deleting inside an array may require shifting $O(n)$ entries.

A LINKED LIST stores a value and a pointer to the next record. A DOUBLY LINKED LIST also stores a pointer to the previous record. Pointers to the first and last records identify the ends. Traversing the list takes $O(n)$ time, and reaching the ith record from the front takes $O(i)$ time. Given the record and neighboring pointers, inserting or deleting takes $O(1)$ time.

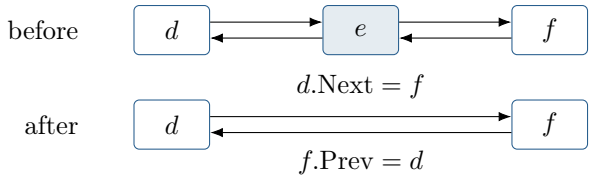


Figure 4: Deleting an interior record e from a doubly linked list. Only its neighbors' pointers change; no later records move.

Insertion reverses the change: link e to both neighbors and redirect each neighbor's pointer to e. At an endpoint, update the first or last pointer. Conversion between a list and an array takes $O(n)$ time, so preprocessing can choose the representation best suited to the operations that follow.

2.3.2 Constant time per proposal

Index men and women separately by $1, \ldots, n$. Store $\text{ManPref}[m, i]$, the ith woman on man m's list, and $\text{WomanPref}[w, i]$, the ith man on woman w's list. Maintain the following additional structures.

1. **Free men.** Keep a linked list of free men and choose the first. Remove him when he becomes engaged; if he displaces another man, insert the displaced man at the front. A rejected man stays free.
2. **Next proposal.** Initialize $\text{Next}[m] = 1$ for every man. His next proposal goes to

   $$w = \text{ManPref}[m, \text{Next}[m]].$$

   Increment $\text{Next}[m]$ after every proposal, accepted or rejected.
3. **Current partner.** Initialize $\text{Current}[w]$ to a null symbol. Later it gives w's partner, or indicates that she is free.
4. **Inverse rankings.** Precompute an array satisfying

   $$\text{Ranking}[w, \text{WomanPref}[w, i]] = i \qquad (1 \le w, i \le n).$$

   Woman w prefers m to m' exactly when $\text{Ranking}[w, m] < \text{Ranking}[w, m']$.

The preference array answers who occupies a rank; the inverse array answers the rank of a specified person. Searching a woman's original list for each comparison would cost $O(n)$ per proposal and yield only an $O(n^3)$ bound. Building all inverse rankings costs $O(n^2)$ once. The other initializations cost $O(n)$, and every proposal then costs $O(1)$. Consequently,

$$T(n) = O(n^2) + O(n^2) \cdot O(1) = O(n^2).$$

The total storage is $O(n^2)$, dominated by the preference and ranking arrays. These representations implement the algorithm proved correct in Chapter 1.

2.4 A survey of common running times

2.4.1 Linear time: one pass and accounting

To find the maximum of $n \geq 1$ numbers, initialize the current maximum to a_1, then replace it by a_i whenever a_i is larger. After processing a_i, the maintained value is $\max\{a_1, \ldots, a_i\}$, so the final value is correct. There is constant work per entry, giving $O(n)$ time and $O(1)$ extra space. Such a pass also fits settings where a stream can be read only once and cannot be stored in full.

Merging two sorted lists shows a subtler way to obtain a linear bound. Let $A = (a_1, \ldots, a_n)$ and $B = (b_1, \ldots, b_n)$ be nondecreasing.

Algorithm 2.1: Merge two sorted lists

```
Input: Sorted lists A and B, each of length n
Set i ← 1, j ← 1, and C ← an empty output list
while i ≤ n and j ≤ n do
    if a_i ≤ b_j then
        Append a_i to C; set i ← i + 1
    else
        Append b_j to C; set j ← j + 1
Append the unconsumed suffix of either list to C
return C
```

The smaller current head is a smallest remaining element, because each input is sorted. Appending it preserves the sorted output; once one list is exhausted, the remaining suffix is already in order. For the book's lists,

$$A = (2, 3, 11, 19), \qquad B = (4, 9, 16, 25),$$
$$C = (2, 3, 4, 9, 11, 16, 19, 25).$$

A head can participate in many comparisons. For even n, take $A = (1, 3, \ldots, 2n-1)$ and $B = (n, n+2, \ldots, 3n-2)$: the first element of B waits through $n/2$ selections from A. Thus it is not correct to claim that every element is compared only a constant number of times.

Instead, charge an iteration to the element it appends. Each element is appended once, so at most $2n$ iterations can be charged. Constant work per iteration, followed by copying the remaining suffix, gives $O(n)$ total time and $O(n)$ output space.

2.4.2 Sorting and $O(n \log n)$ time

Mergesort divides the input into two halves, recursively sorts each, and merges them in linear time. Balanced splitting gives $O(\log n)$ levels with $O(n)$ total work per level, yielding $O(n \log n)$ time. The detailed recursion analysis comes with divide and conquer.

Another common pattern is sorting followed by a linear scan. Given $n \geq 2$ file-arrival times, first sort them into $x_{(1)} \leq \cdots \leq x_{(n)}$. The longest gap with no arrival between the first and last arrivals has length

$$\max_{1 \leq i < n} \bigl(x_{(i+1)} - x_{(i)}\bigr).$$

An arrival-free gap cannot cross an intermediate arrival, so checking consecutive pairs suffices. Sorting costs $O(n \log n)$ and the scan costs $O(n)$, for $O(n \log n)$ overall.

2.4.3 Pairs, nested loops, and fixed-size subsets

Quadratic time. For the closest pair among $n \geq 2$ planar points, enumerate the $\binom{n}{2} = n(n-1)/2$ distinct pairs. Compute

$$d_{ij} = \sqrt{(x_i - x_j)^2 + (y_i - y_j)^2}$$

and retain a pair of minimum distance. Under the constant-time arithmetic model, each pair costs $O(1)$, so this brute-force algorithm takes $O(n^2)$ time. Two loops over $i < j$ implement the enumeration. The number of candidate pairs does not prove that every algorithm must inspect them all; later methods improve this bound.

Cubic time. Suppose $S_1, \ldots, S_n \subseteq \{1, \ldots, n\}$, and ask whether two distinct sets are disjoint. Assume we can list a set's elements in constant time per element and test membership in constant time. For each of $O(n^2)$ pairs (S_i, S_j), inspect the at most n elements of S_i and test whether any belongs to S_j. A pair is disjoint exactly when none does. This gives $O(n^3)$ time. The bound depends on the stated representation and membership cost.

A fixed exponent k. To test for an independent set of a fixed size k in an n-node graph, enumerate all k-node subsets and check each pair of their nodes for an edge. Assume an adjacency test costs $O(1)$. There are

$$\binom{n}{k} = \frac{n(n-1)\cdots(n-k+1)}{k!} \leq \frac{n^k}{k!}$$

candidates and $O(k^2)$ work per candidate, giving $O(k^2\binom{n}{k}) \subseteq O(k^2 n^k)$ time. For fixed $k \geq 2$, this is $O(n^k)$ and hence polynomial in n. If k is part of the input, the exponent is no longer fixed, so the same argument does not establish a polynomial-time algorithm. The cases $k = 0, 1$ are immediate.

2.4.4 Exponential and factorial search spaces

To find a maximum independent set by brute force, consider every node subset. Each of the n nodes is either included or excluded, producing 2^n subsets. Testing all pairs within each candidate costs at most $O(n^2)$, so this algorithm takes $O(n^2 2^n)$ time.

The same 2^n candidate count occurs in interval scheduling, but a greedy algorithm solves that problem in $O(n \log n)$ time. A large search space is a property of the formulation, not by itself a lower bound on all algorithms for the problem.

Factorials arise from assignments and orderings:

- Pairing n men with n women gives $n(n-1)\cdots 1 = n!$ perfect matchings. Gale–Shapley avoids enumerating them.
- A traveling-salesman tour with a fixed starting city can be specified by ordering the remaining $n-1$ cities, giving $(n-1)!$ candidate orders. Evaluating each order also takes time, so the candidate count alone is not the complete running time.

Unlike stable matching and interval scheduling, maximum independent set and traveling salesman are believed to have no polynomial-time algorithms.

2.4.5 Sublinear time: binary search

An algorithm cannot read all n input entries in sublinear time. Such bounds require a model that permits direct queries and a task that does not require reading every entry.

For a sorted array $A[1..n]$, binary search maintains an interval $[\ell, r]$ that contains every possible occurrence of the target p. Initially $\ell = 1$ and $r = n$. Probe $m = \lfloor (\ell + r)/2 \rfloor$:

- If $A[m] = p$, return that position.
- If $A[m] < p$, set $\ell = m + 1$.
- If $A[m] > p$, set $r = m - 1$.

Sorted order justifies each discarded half. If $\ell > r$, report that p is absent. After t unsuccessful probes, at most $n/2^t$ entries remain; $O(\log n)$ probes reduce this to constant size. Each probe takes $O(1)$ time in an array, so the search takes $O(\log n)$ time. Sorting or loading the input is outside this query bound.

2.5 Priority queues and heaps

2.5.1 The operations and simpler representations

A PRIORITY QUEUE maintains a changing set S of items. Each item v has a key $\text{key}(v)$; smaller keys mean higher priority. We want to insert items, identify or extract a minimum-key item, delete items, and change priorities. For example, a scheduler can insert newly arriving processes and extract the most urgent active process.

Some simple representations make one operation fast at the expense of another:

- An unsorted list with a pointer to a minimum supports insertion and minimum lookup in $O(1)$ time. Extracting that minimum may require an $O(n)$ scan to find the next one.
- A sorted array allows binary search for an insertion position, but shifting entries still costs $O(n)$ time.
- A sorted linked list allows constant-time insertion at a known position, but finding that position may take $O(n)$ time.

A heap instead supports insertion and extraction in $O(\log n)$ time, with $O(1)$ minimum lookup.

2.5.2 Heap order and array representation

A BINARY MIN-HEAP is a binary tree filled level by level from left to right, satisfying

$$\text{key}(\text{parent of } v) \le \text{key}(v) \quad \text{for every nonroot item } v.$$

Following parent links shows that the root has a minimum key. Heap order does not require siblings, or all entries of a level, to be sorted.

If a capacity N is known, allocate an array $H[1..N]$. With n current items, occupy exactly positions $1, \dots, n$. The root is $H[1]$, and

$$\text{parent}(i) = \lfloor i/2 \rfloor \quad (i > 1), \qquad \text{left}(i) = 2i, \qquad \text{right}(i) = 2i + 1,$$

where a child exists only if its index is at most n. The tree has height $\lfloor \log_2 n \rfloor$ for $n \ge 1$.

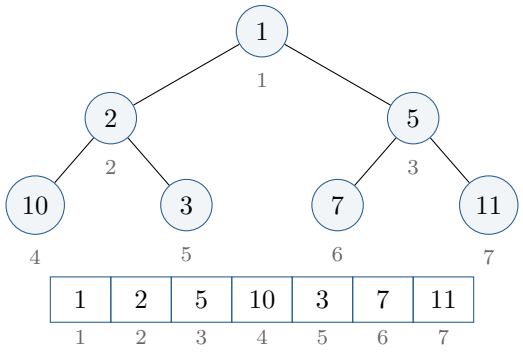


Figure 5: The first three levels of the book's heap, shown as a seven-item heap and its array. Large numbers are keys; small numbers are positions. The array is in heap order, although 10 precedes 3.

2.5.3 Insertion and heapify-up

To insert v when $n < N$, put it at $H[n+1]$ and increase n. This preserves the tree's shape. Only the new item's key may be too small relative to its parent. Move it upward while that inequality is violated.

Algorithm 2.2: Heapify-up at position i

```
while i > 1 do
    Set j ← ⌊i/2⌋
    if key(H[j]) ≤ key(H[i]) then
        return
    Swap H[i] and H[j]
    Set i ← j
```

Proposition 2.4 [Upward repair]. *Suppose increasing the key at position i to some value α would make H a heap. Heapify-up restores heap order in $O(1 + \log i)$ time.*

Proof. Write $v = H[i]$. If $i = 1$, its key is already no larger than its children's, so H is a heap. Otherwise let its parent be $w = H[j]$ with key β. If $\text{key}(v) \geq \beta$, all edges satisfy heap order and no swap is needed.
Otherwise $\text{key}(v) < \beta \leq \alpha$. Swap v with w. At the lower position, w fits above the children: their keys were at least α, and hence at least β. If we temporarily raised v's key at j to β, the whole array would be a heap. Thus the same hypothesis now holds at j, strictly closer to the root. Induction proves correctness. Each step costs $O(1)$ and halves the index, giving $O(1 + \log i)$ time. □

For insertion, making the new leaf's key sufficiently large would restore heap order, so the proposition applies. Insertion takes $O(\log n)$ time for $n \geq 2$, and constant time for an empty or one-item heap.

Example: In the book's upward-repair example, a new key 3 occupies position 15. Its ancestors at positions $7, 3, 1$ have keys $11, 5, 2$. Swap 3 with 11, then with 5, and stop below 2. The changed path is

position	1	3	7	15
before	2	5	11	3
after	2	3	5	11

Only that rootward path changes; the other entries keep their positions.

2.5.4 Deletion and heapify-down

To delete position i from a heap of n items, move the final item $H[n]$ into the hole and reduce the length to $n-1$. If $i = n$, simply remove the final item and stop. Otherwise the replacement may be too small for its parent or too large for a child.

If it is smaller than its parent, use heapify-up. Otherwise use heapify-down, which always compares with the smaller-key child.

Algorithm 2.3: Heapify-down at position i

```
1  Set n ← length(H)
2  while 2i ≤ n do
3  |  Set j ← 2i
4  |  if 2i + 1 ≤ n then
5  |  |  if key(H[2i + 1]) < key(H[2i]) then
6  |  |  |  Set j ← 2i + 1
7  |  if key(H[i]) ≤ key(H[j]) then
8  |  |  return
9  |  Swap H[i] and H[j]
10 |  Set i ← j
```

Proposition 2.5 [Downward repair]. *Suppose decreasing the key at position i to some value α would make H a heap. Heapify-down restores heap order in $O(\log n)$ time for $n \geq 2$.*

Proof. At a leaf, the key is already large enough for its parent, so the heap is valid. At an internal node, let $v = H[i]$ and choose a smaller-key child $w = H[j]$. If $\text{key}(v) \leq \text{key}(w)$, no edge is violated. Otherwise swap them.
The promoted w is no larger than any other child, so the order below position i is repaired. It is large enough for the parent of i, since its key is at least α, which was large enough for that parent. At position j, decreasing v's key to the old key of w would make the whole array a heap. Thus the hypothesis moves down one level and induction applies. Each swap costs $O(1)$; there are at most $\lfloor \log_2 n \rfloor$ levels to traverse. □

After replacing a deleted item, all edges away from position i remain valid. If the replacement is smaller than its parent, it is also smaller than its children, because the old parent was no larger than

the deleted item and its children. This is the upward-repair case. Otherwise its parent edge is valid, and it can only need downward repair. Deletion therefore takes $O(\log n)$ time.

Example: In the book's downward-repair example, key 21 at position 3 has children with keys 7 and 11. Swap with 7 at position 6. Its new children have keys 15 and 8, so swap with 8 at position 13 and stop at that leaf. The path changes from $(21, 7, 8)$ to $(7, 8, 21)$. Choosing the smaller child is essential: promoting 11 first would leave 7 below a larger parent.

2.5.5 Access by item and operation bounds

To delete or update an item v by name, maintain Position$[v]$, its current array index. Every swap updates the positions of both items in $O(1)$ time. Moving the final item into a deletion hole also updates its position. Insertion records the new position, and deletion invalidates the removed item's position.

To change v's key to α, find $i =$ Position$[v]$, save the old key, and assign the new one. If it decreases, use heapify-up; if it increases, use heapify-down. Equal keys need no repair. The position array requires constant-time indexing by item identifier, as with graph nodes numbered in advance.

Operation	Method	Time
StartHeap(N)	Initialize capacity-N storage	$O(N)$
FindMin(H)	Read the root	$O(1)$
Insert(H, v)	Append, then repair upward	$O(\log n)$
Delete(H, i)	Replace by last item, then repair	$O(\log n)$
ExtractMin(H)	Save root, then delete position 1	$O(\log n)$
ChangeKey(H, v, α)	Locate, change key, then repair	$O(\log n)$

Here n is the current size; logarithmic bounds include constant time at sizes 0 and 1. Minimum lookup and extraction require a nonempty heap, and deletion requires a valid item or index. Capacity is N, so the array uses $O(N)$ space; the position array additionally covers the item identifiers.

Priority queues also yield a sorting algorithm: insert n numbers using their values as keys, then extract the minimum n times. The output is nondecreasing, and these $2n$ operations take $O(n \log n)$ time, including $O(n)$ initialization when capacity is n. Comparison-based sorting needs $\Omega(n \log n)$ comparisons in the worst case. Thus this complete sequence of priority-queue operations cannot take less than that total time in the comparison model. This does not require every individual operation to have an $\Omega(\log n)$ cost.

Source. Chapter 2, Basics of Algorithm Analysis, *Algorithm Design.*

Chapter 3: Graphs

A graph records pairwise relationships. Once a problem is expressed in this form, graph search can answer questions about reachability, shortest paths, grouping, and dependencies. The representation and the order of exploration determine both the running time and the structure revealed by the search.

3.1 Basic definitions and applications

3.1.1 Nodes, edges, and models

An UNDIRECTED GRAPH $G = (V, E)$ consists of a set of nodes, or vertices, and a set of edges. An edge $\{u, v\}$ joins its two ends and represents a symmetric relationship. In a DIRECTED GRAPH, an edge (u, v) goes from its tail u to its head v; reversing the pair changes the edge. Unless stated otherwise, graphs here have no self-loops or duplicate edges, and "graph" means an undirected graph. Opposite directed edges (u, v) and (v, u) may both be present.

A graph model should specify what its nodes and edges mean. The chapter's applications include the following.

Setting	Nodes	Meaning of an edge
Transportation	Airports or stations	A direct flight or rail connection
Communication	Devices or providers	A physical link, reception of a signal, or a peering agreement
Information	Web pages or papers	A hyperlink or citation to another item
Social relations	People or organizations	Friendship, advice, or membership
Dependencies	Courses, tasks, or modules	A prerequisite, required output, or function call

Direction is part of the model: a hyperlink need not be reciprocated, and one radio may receive another's signal without being able to send a signal back. A membership graph can be bipartite, with people on one side and organizations on the other. Some edges represent physical connections; others represent abstract relationships.

3.1.2 Paths, distance, and trees

A PATH is a sequence $v_0, v_1, \ldots, v_k$ in which consecutive nodes are joined by an edge. Its length is its number k of edges. A path is SIMPLE if its nodes are distinct. A CYCLE returns to its starting

node, has no other repeated node, and does not reuse an edge. In a simple undirected graph, a cycle has at least three edges. Directed paths and cycles must follow every edge in its given direction.

An undirected graph is CONNECTED if every pair of nodes has a path between them. The DISTANCE $\text{dist}(u, v)$ is the minimum number of edges in a u-v path, or ∞ if no such path exists. In a directed graph, reachability need not be symmetric; the graph is STRONGLY CONNECTED if every node can reach every other node.

A TREE is a connected undirected graph with no cycle. There is exactly one simple path between any two of its nodes: two distinct simple paths would contain a cycle. Deleting an edge disconnects a tree, since another route between that edge's ends would also form a cycle.

Choose a root r to give a tree a hierarchy. Each nonroot node's PARENT is its predecessor on the path from r; it is a CHILD of that parent. Nodes on a root-to-node path are that node's ANCESTORS, and nodes below them are their DESCENDANTS. A LEAF has no children. Rooting changes the hierarchy and drawing, but not which nodes are joined.

Proposition 3.1 [EDGES IN A TREE]. *Every tree with n nodes has exactly $n - 1$ edges.*

Proof. Root the tree at any node. Each of the $n - 1$ nonroot nodes has one edge to its parent, and every edge is counted by exactly one such node. □

More generally, for an undirected graph on $n \geq 1$ nodes, any two of these three properties imply the third:

1. The graph is connected.
2. The graph has no cycle.
3. The graph has exactly $n - 1$ edges.

3.2 Connectivity and graph traversal

Given nodes s, t, the connectivity problem asks whether an s-t path exists. A search from s can build the entire set R of reachable nodes. This is the CONNECTED COMPONENT containing s.

3.2.1 Growing a component

Start with $R = \{s\}$. Whenever an edge $\{u, v\}$ has $u \in R$ and $v \notin R$, add v. Stop when no edge leads from R to its complement.

Proposition 3.2 [Reachability by exploration]. *This procedure finds exactly the connected component containing s, regardless of which eligible edge is chosen at each step.*

Proof. Every added node is reachable: extend a path to u by the edge $\{u, v\}$. For the converse, suppose a reachable node w is absent when the procedure stops. On an s-w path, let v be the first node outside R and let u immediately precede it. Then $u \in R$ and $\{u, v\}$ is an edge leaving R, contradicting the stopping rule. □

When v is added, record the node u responsible for discovering it as its parent. Parent links go to nodes added earlier, so they form a tree rooted at s. Following parents from t back to s and reversing the sequence recovers an actual path whenever t is reachable.

3.2.2 Breadth-first search

Breadth-first search, or BFS, explores in layers:

$$L_0 = \{s\}, \qquad L_{i+1} = \{v \text{ not in earlier layers} : \{u, v\} \in E \text{ for some } u \in L_i\}.$$

Stop when the next layer is empty. Each newly discovered node chooses a parent in the preceding layer, producing a BFS tree.

Theorem 3.3 [BFS layers and shortest paths]. *For each $i \geq 0$, L_i consists exactly of the nodes at distance i from s. The tree path to a node in L_i is a shortest path of length i.*

Proof. Use induction on i. The statement holds for L_0. If $v \in L_{i+1}$, its parent is in L_i, so extending the parent's tree path gives a path of length $i + 1$. A shorter path would put v in an earlier layer by the induction hypothesis, contrary to its being new.
Conversely, if $\operatorname{dist}(s, v) = i + 1$, take a shortest path and let u precede v. Its prefix to u must be shortest, so $u \in L_i$. The node v has not appeared earlier and will be added when u is scanned. Thus $v \in L_{i+1}$. □

In the book's example, the component of node 1 is $\{1, \ldots, 8\}$. Its layers are

$$L_0 = \{1\}, \quad L_1 = \{2, 3\}, \quad L_2 = \{4, 5, 7, 8\}, \quad L_3 = \{6\}.$$

The other components, $\{9, 10\}$ and $\{11, 12, 13\}$, are not reached. Scanning 2 before 3 makes 2 the parent of $4, 5$, while 3 becomes the parent of $7, 8$; node 5 then discovers 6.

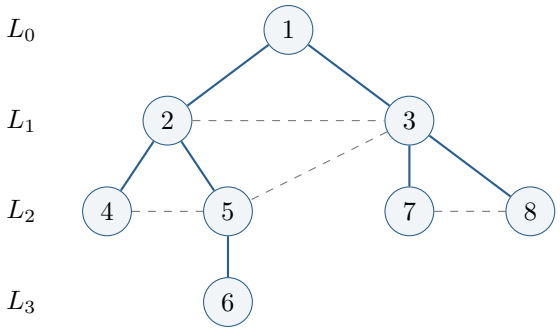


Figure 6: BFS on the book's eight-node component, starting at 1. Solid edges form the search tree; dashed edges belong to the graph but discover no new node. Horizontal rows give distance from 1.

Proposition 3.4 [Edges between BFS layers]. *If $\{x, y\} \in E$ with $x \in L_i$ and $y \in L_j$, then $|i - j| \leq 1$.*

Proof. A shortest path to x, followed by $\{x, y\}$, shows $j \leq i + 1$. Reversing the roles of x and y gives $i \leq j + 1$. □

Thus an undirected edge joins nodes in the same or consecutive layers. The parent tree may depend on the order of scanning neighbors, but the layer numbers do not.

3.2.3 Depth-first search

Depth-first search, or DFS, follows an unexplored neighbor as far as possible, then backtracks when no unexplored neighbor remains. Initially mark every node unexplored and set $R = \varnothing$. Invoke the following recursive procedure at s.

Algorithm 3.1: Recursive DFS at node u

```
1 Mark u explored and add u to R
2 for each v ∈ Adj[u] do
3     if v is unexplored then
4         Set parent[v] ← u
5         Invoke DFS(v)
```

The discovery edges form a DFS tree. In the same component, the book's execution first follows $1, 2, 3, 5, 4$. It returns to 5 and visits 6, returns to 3 and visits 7, then reaches 8 from 7. Its discovery order is $1, 2, 3, 5, 4, 6, 7, 8$. The route $1, 2, 3$ to node 3 is not shortest: the graph also has edge $\{1, 3\}$.

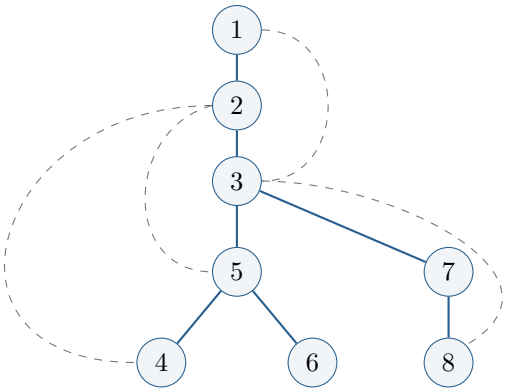


Figure 7: A DFS tree of the same graph, with the same solid/dashed edge convention. Every dashed edge joins an ancestor to a descendant, unlike the edges between separate branches that are possible in BFS.

Proposition 3.5 [NONTREE EDGES IN UNDIRECTED DFS]. *Every graph edge outside the DFS tree joins an ancestor to a descendant.*

Proof. All nodes first explored during a call DFS(x) become descendants of x: they are reached by that call or its recursive descendants. Now consider a nontree edge $\{x, y\}$ and suppose x is explored first. Before DFS(x) finishes, it examines this edge. Since the edge does not discover y, y must already be explored at that moment. It was not explored when DFS(x) began, so it was explored during the call and is a descendant of x. □

This property uses an undirected graph. It is not a restriction on all nontree edges of a directed DFS tree.

3.2.4 All connected components

The components of two nodes are either identical or disjoint. If s and t have a path between them, concatenate paths through that path to show that every node reachable from either is reachable from both. If their components overlap at a node v, paths from s to v and from v to t likewise connect s and t. Thus overlap forces equality.

To find all components, start a search at any unvisited node, label every node it reaches with that component, and repeat until all nodes are visited. The resulting search trees form a forest, with one root per component.

3.3 Implementing graph traversal

Write $n = |V|$ and $m = |E|$, and index nodes by $1, \ldots, n$. Keep both parameters in time bounds: a simple undirected graph has $0 \le m \le \binom{n}{2}$, so the relationship between m and n varies. For a connected graph, $m \ge n - 1$, and $O(n + m)$ can also be written $O(m)$ when $n \ge 2$.

3.3.1 Adjacency matrices and adjacency lists

An ADJACENCY MATRIX is an $n \times n$ array A, with $A[u, v] = 1$ exactly when $\{u, v\} \in E$. It is symmetric for an undirected graph. An ADJACENCY LIST stores, for each u, a list Adj$[u]$ of its neighbors. In an undirected graph, every edge appears twice, once at each end.

Let n_v denote the DEGREE of node v, the number of incident edges. Counting each edge once at each end gives

$$\sum_{v \in V} n_v = 2m.$$

This identity is the main accounting step behind linear-time traversal.

Operation or cost	Adjacency matrix	Adjacency lists
Storage	$\Theta(n^2)$	$\Theta(n + m)$
Test whether $\{u, v\} \in E$	$O(1)$	$O(1 + n_u)$
List all neighbors of u	$\Theta(n)$	$O(1 + n_u)$

Lists use space for n list headers and $2m$ entries. They are especially useful for SPARSE GRAPHS, where m is much smaller than n^2. An edge test may scan the list, but a traversal reads each neighbor in constant time. A matrix scans an entire row even for a low-degree node.

3.3.2 Queues and a BFS implementation

A QUEUE removes items in first-in, first-out order (FIFO). A STACK removes items in last-in, first-out order (LIFO). Both can use linked lists with constant-time insertion and removal: remove from the front; insert at the back for a queue and at the front for a stack. A queue needs access to both ends.

For BFS, mark a node discovered as soon as it is added to the queue. An array d stores distances, and parent links store shortest paths.

Algorithm 3.2: BFS from s using one queue

```
for each v ∈ V do
    Set Discovered[v] ← false, d[v] ← ∞, and parent[v] ← null
Set Discovered[s] ← true and d[s] ← 0
Initialize an empty queue Q and enqueue s
while Q is not empty do
    Dequeue its first node u
    for each v ∈ Adj[u] do
        if Discovered[v] = false then
            Set Discovered[v] ← true
            Set d[v] ← d[u] + 1 and parent[v] ← u
            Enqueue v
```

The queue processes all of L_i before L_{i+1}, because newly discovered nodes are appended after the nodes waiting to be processed. It therefore implements the layered search. Alternatively, maintain separate lists for each layer; within one layer, any processing order is valid.

Proposition 3.6 [Traversal time]. *BFS and DFS can each be implemented in $O(n+m)$ time using adjacency lists.*

Proof. For BFS, initialization costs $O(n)$. Each node enters and leaves the queue at most once. Exploring u scans its n_u neighbors, so all scans cost

$$O\left(\sum_{u \in V} n_u\right) = O(m).$$

The total is $O(n+m)$. Recursive DFS likewise marks each node once and scans its adjacency list once; the same sum applies. □

BFS uses $O(n)$ extra storage for the queue and arrays. Recursive DFS uses $O(n)$ extra storage, including its call stack. With an adjacency matrix, scanning rows can instead take $O(n^2)$ time.

3.3.3 An explicit stack for DFS

The book's iterative DFS puts neighbors on a stack, but marks a node explored only when it is first popped. A node can have several pending copies. Store each stack entry as a pair (v, p), where p is the node that caused that copy to be pushed; accept its parent only on the first successful pop.

Algorithm 3.3: DFS using an explicit stack

```
1 Mark every node unexplored and set every parent to null
2 Initialize stack S with the single entry (s, null)
3 while S is not empty do
4 |   Pop its top entry (u, p)
5 |   if u is unexplored then
6 |   |   Mark u explored and set parent[u] ← p
7 |   |   for each v ∈ Adj[u], in list order do
8 |   |   |   Push (v, u) onto S
```

Since the last neighbor pushed is considered first, this visits nodes in the same order as recursive DFS with every adjacency list reversed. Pending neighbors of an older node remain below the entries created by the current exploration. Thus the search finishes the newer branch first. Marking all neighbors explored when they are pushed would change this behavior and can give a different parent tree.

Each adjacency list is scanned only on the first successful pop. In the component reached from s, every edge causes two pushes, in addition to the initial entry for s. There are at most $1+2m$ pushes and the same number of pops, even though a node may appear repeatedly. Thus this implementation

also takes $O(n+m)$ time. Its explicit stack can use $O(n+m)$ space; it need not have only one entry per node.

When finding all components, initialize visitation arrays once and run an outer loop over all nodes, starting a search only at an unvisited node. Each node and edge is then processed in its own component, so all searches together take $O(n+m)$ time. Reinitializing all n flags separately for every component would lose this bound.

3.4 Testing bipartiteness

A graph is BIPARTITE if its nodes split into two sets X, Y and every edge has one end in each. Equivalently, color nodes red and blue so that every edge has differently colored ends.

An odd cycle cannot be colored this way: alternating colors around its edges returns to the starting color one edge too early, leaving its last edge with two equal colors. Thus a bipartite graph contains no odd cycle. The converse follows from BFS.

3.4.1 Coloring by layer parity

In each connected component, choose a root and run BFS. Color even layers red and odd layers blue, then scan all edges for a same-color pair. The layer and edge scans together take $O(n+m)$ time. An isolated node may receive either color.

Theorem 3.7 [BIPARTITENESS AND ODD CYCLES]. *For a BFS layering of a connected graph, exactly one of the following holds:*

1. *No edge joins two nodes in the same layer. The layer-parity coloring is a valid bipartition.*
2. *Some edge joins nodes in the same layer. The graph contains an odd cycle and is not bipartite.*

Consequently, a graph is bipartite if and only if it has no odd cycle.

Proof. Every edge joins the same or consecutive BFS layers. If there is no edge within a layer, every edge joins consecutive layers and hence opposite colors.
Otherwise let $\{x, y\}$ be an edge with $x, y \in L_j$. In the BFS tree, let $z \in L_i$ be their LOWEST COMMON ANCESTOR, the common ancestor farthest from the root. Since $x \neq y$ have the same depth, $i < j$. The tree paths from z to x and y share only z; otherwise there would be a lower common ancestor. Combining these two paths with $\{x, y\}$ gives a simple cycle of length

$$(j-i)+1+(j-i) = 2(j-i)+1,$$

which is odd. No valid two-coloring can exist. □

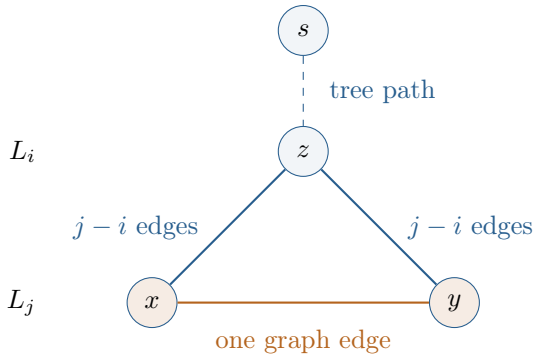


Figure 8: Schematic of the odd-cycle proof. Each sloping segment represents an entire tree path. Their only shared node is z, so adding the edge $\{x, y\}$ closes a cycle with an odd number of edges.

The proof also gives a way to exhibit the obstruction: follow the two parent chains to their first common ancestor, discard their common prefix toward the root, and join the remaining paths with the conflicting edge. In the traversal example, nodes 2 and 3 lie in L_1 and are adjacent, yielding the triangle $1, 2, 3, 1$.

3.5 Connectivity in directed graphs

3.5.1 Forward and reverse searches

For a directed graph, keep outgoing and incoming adjacency lists. An edge (u, v) puts v on u's outgoing list and u on v's incoming list. Each family of lists has m entries, so total storage is $O(n + m)$.

BFS and DFS follow outgoing edges to find nodes reachable from a source s, still in $O(n + m)$ time. Directed BFS layers give shortest directed path lengths. The undirected restriction $|i - j| \leq 1$ need not hold for a directed edge from L_i to L_j: a backward edge may go many layers toward the source. Only $j \leq i + 1$ is guaranteed for reachable endpoints.

To find nodes that can reach s, reverse every edge to form G^{rev} and search from s there. Following the incoming lists directly implements this reverse search. A path $s \rightsquigarrow v$ in G^{rev} corresponds exactly to a path $v \rightsquigarrow s$ in G.

3.5.2 Strong connectivity and strong components

Nodes u, v are MUTUALLY REACHABLE if both $u \rightsquigarrow v$ and $v \rightsquigarrow u$. This relation is transitive: if u, v are mutually reachable and v, w are mutually reachable, concatenate the appropriate paths through v to obtain both $u \rightsquigarrow w$ and $w \rightsquigarrow u$.

To test strong connectivity in a nonempty directed graph:

1. Choose any node s and search from it in G.
2. Search from the same s in G^{rev}.
3. Accept exactly when both searches reach every node.

Proposition 3.8 [TWO-SEARCH TEST]. *This test determines strong connectivity in $O(n+m)$ time.*

Proof. In a strongly connected graph, both searches must reach every node. Conversely, if both succeed, any two nodes u, v have a path $u \rightsquigarrow s \rightsquigarrow v$. Hence every node reaches every other node. Each search and the construction or use of reverse adjacency lists costs $O(n+m)$ time. □

Let $F(s)$ be the nodes reached by the forward search, and $B(s)$ those reached by the reverse search. The STRONG COMPONENT containing s is exactly

$$C(s) = F(s) \cap B(s).$$

A forward search alone can reach nodes that cannot return to s.

Strong components are either identical or disjoint. If s, t are mutually reachable, transitivity shows that $C(s) = C(t)$. If they are not mutually reachable, an overlapping node v would imply $s \rightsquigarrow v \rightsquigarrow t$ and $t \rightsquigarrow v \rightsquigarrow s$, a contradiction. Thus strong components partition the graph. There can be directed edges between distinct strong components, unlike the absence of edges between distinct undirected components.

All strong components can also be found in $O(n+m)$ total time, but repeating two full searches independently for each node does not establish that bound.

3.6 Directed acyclic graphs and topological ordering

A DIRECTED ACYCLIC GRAPH, or DAG, has no directed cycle. It need not look like a tree: on nodes $1, \ldots, n$, including every edge (i, j) with $i < j$ gives $\binom{n}{2}$ edges and no directed cycle.

DAGs model dependencies. If an edge (u, v) means that task u must finish before task v starts, a directed cycle makes those requirements impossible: no task on the cycle can be the first to finish.

Definition 3.9 [TOPOLOGICAL ORDERING]. A TOPOLOGICAL ORDERING of a directed graph is an ordering $v_1, \ldots, v_n$ of its nodes such that every edge (v_i, v_j) satisfies $i < j$. Every edge points forward in the ordering.

Proposition 3.10 [An ordering certifies acyclicity]. *If a directed graph has a topological ordering, then it is a DAG.*

Proof. If there were a directed cycle, choose its lowest-indexed node v_i. Its predecessor v_j on the cycle has $j > i$, so the edge (v_j, v_i) points backward, contradicting the ordering. □

3.6.1 Finding a first node

The first node in a topological ordering must have no incoming edge. The following lemma ensures that such a choice is always available in a DAG.

Lemma 3.11 [A source exists]. *Every nonempty DAG has a node with no incoming edge.*

Proof. Suppose instead that every node has an incoming edge. Starting at any v_0, repeatedly choose v_{i+1} with an edge (v_{i+1}, v_i). The process cannot stop. Among the first $n + 1$ visited nodes, some node repeats. At the first repetition, the intervening distinct nodes form a directed cycle when read in the direction of the edges. This contradicts acyclicity. □

Theorem 3.12 [DAGs admit topological orderings]. *A directed graph has a topological ordering if and only if it is a DAG.*

Proof. The forward implication was proved above. For the converse, induct on the number of nodes. The empty graph has an empty ordering. In a nonempty DAG, choose a node v with no incoming edge and put it first. Removing v cannot create a cycle, so the remaining graph is a smaller DAG and has a topological ordering by induction. Append that order after v. Edges within the remainder point forward, as do all edges incident to v. □

This proof gives an algorithm: repeatedly choose a node with no incoming edge, output it, and delete it. At each step a DAG still has a valid choice. Repeated scans for an eligible node can give $O(n^2)$ time; maintaining the right counts makes the same high-level algorithm linear.

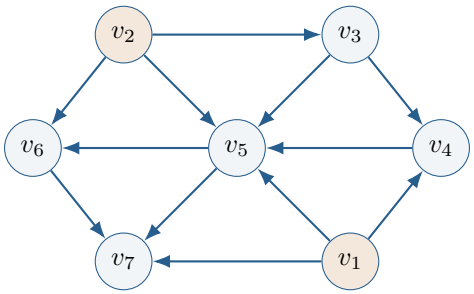


Figure 9: The book's DAG, with its topological labels. Each arrow goes from a lower subscript to a higher one, even when it points left or upward in the drawing. The shaded nodes v_1, v_2 initially have no incoming edges.

Here either v_1 or v_2 may be chosen first. Choosing v_1 and then v_2 makes v_3 eligible; continuing yields $v_1, v_2, v_3, v_4, v_5, v_6, v_7$. In general, different eligible choices can give different valid orderings.

3.6.2 Maintaining incoming-edge counts

Call a node active until it is output. Maintain $\mathrm{In}[w]$, the number of incoming edges to w from active nodes, and a set S of active nodes whose count is zero. A linked-list queue or stack supports the needed insertion and removal in constant time.

Algorithm 3.4: Topological ordering with incoming-edge counts

```
Initialize In[w] ← 0 for every w ∈ V
for each directed edge (u, w) ∈ E do
    Set In[w] ← In[w] + 1
Initialize S to all nodes with In[w] = 0 and A to an empty list
while S is not empty do
    Remove any node v from S and append it to A
    for each w on the outgoing list of v do
        Set In[w] ← In[w] − 1
        if In[w] = 0 then
            Add w to S
if |A| = n then
    return A
else
    Report that the graph is not a DAG
```

Removing v deletes exactly its outgoing edges from the active graph, so decrementing its neighbors' counts preserves the invariant. A node enters S when its last active predecessor is removed, and all its predecessors therefore appear earlier in A. In a DAG, the source lemma guarantees that S cannot be empty while active nodes remain. If this happens on an arbitrary directed graph, the remaining graph has every incoming count positive and hence contains a directed cycle.

Initialization scans all nodes and edges. Each node is output at most once, and each edge causes one decrement when its tail is output. The algorithm takes $O(n + m)$ time and $O(n)$ extra storage beyond the graph. The graph need not be physically changed; the counts represent deletion.

Source. Chapter 3, Graphs, *Algorithm Design*.